

Collection of Pseudo-Code definitions for the Tensor Omics Implementation

Asis Hallab, “el Bobito”

April 28, 2025

1 F42 LOESS Module Pseudocode

1.1 Module Overview

The LOESS module provides local weighted regression smoothing for 1D data, specifically for fitting standard deviation as a function of mean expression. It is designed to comply fully with the F42 DataStructure principles: statelessness, preallocated memory, reproducibility, and serialization-readiness.

1.2 Type Definitions

- `type :: LOESS_Model`
 - `real, dimension(:), allocatable :: x_sorted` (sorted predictor values)
 - `real, dimension(:), allocatable :: y_sorted` (sorted response values)
 - `integer :: n` (number of data points)
 - `real :: bandwidth_fraction` (fraction of points used for local fitting, e.g., 0.3)

1.3 Public Procedures

- `subroutine loess_fit(x, y, bandwidth, model, work_int, work_real)`
 - Inputs:
 - * `real, dimension(n) :: x` (predictor values)
 - * `real, dimension(n) :: y` (response values)
 - * `real :: bandwidth` (bandwidth fraction, $0 < \text{bandwidth} \leq 1$)
 - Outputs:
 - * `type(LOESS_Model) :: model`
 - Working memory (preallocated by caller):
 - * `integer, dimension(n) :: work_int`
 - * `real, dimension(n) :: work_real`
- `subroutine loess_predict(model, x_query, y_pred, work_int, work_real)`
 - Inputs:
 - * `type(LOESS_Model) :: model`
 - * `real :: x_query` (query predictor value)
 - Outputs:
 - * `real :: y_pred` (predicted response value)
 - Working memory (preallocated by caller):
 - * `integer, dimension(model%n) :: work_int`
 - * `real, dimension(model%n) :: work_real`

1.4 Algorithm: `loess_fit`

1. Copy input `x` and `y` into temporary arrays using `work_real`.
2. Sort `x` ascendingly, rearranging `y` in the same order (store into `model%x_sorted` and `model%y_sorted`).
3. Store `n = size(x)` and `bandwidth_fraction = bandwidth` into `model`.

1.5 Algorithm: `loess_predict`

1. Perform binary search on `model%x_sorted` to find the insertion point for `x_query`.
2. Expand left and right to collect k nearest neighbors, where

$$k = \max(3, \lfloor \text{bandwidth_fraction} \times n \rfloor)$$

3. Compute distances of collected neighbors to `x_query`.
4. Assign weights using the tricube kernel:

$$w_i = \left(1 - \left(\frac{|x_i - x_{\text{query}}|}{d_{\max}} \right)^3 \right)^3$$

where $d_{\max}$ is the largest distance among the collected neighbors.

5. Solve the weighted least squares problem to fit $y = \beta_0 + \beta_1 x$ locally:

$$\min_{\beta_0, \beta_1} \sum_i w_i (y_i - \beta_0 - \beta_1 x_i)^2$$

(This can be solved by forming normal equations manually or using a tiny LAPACK call if needed.)

6. Predict the response:

$$y_{\text{pred}} = \beta_0 + \beta_1 x_{\text{query}}$$

1.6 Memory Requirements

- During fitting:
 - `2n` reals (temporary storage for sorting)
 - `n` integers (for sorting indices if needed)
- During prediction:
 - `n` reals (for temporary neighbor distances and weights)
 - `n` integers (for neighbor indices if needed)

1.7 Unit Tests for LOESS Module

The following unit tests are designed to verify the correctness, stability, and reproducibility of the LOESS fitting and prediction routines. All tests assume that memory management (allocation of working arrays) is properly handled by the test harness.

1.7.1 Test 1: Exact Interpolation with Noiseless Linear Data

- Input:
 - $\mathbf{x} = [1.0, 2.0, 3.0, 4.0, 5.0]$
 - $\mathbf{y} = [2.0, 4.0, 6.0, 8.0, 10.0]$
 - `bandwidth` = 1.0 (use all points)
- Action:
 - Fit LOESS model.
 - Predict at all original $\mathbf{x}$ points.
- Expected:
 - Predictions exactly match $\mathbf{y}$.
 - Maximum absolute prediction error $< 10^{-12}$.

1.7.2 Test 2: Locality with Bandwidth Restriction

- Input:
 - $\mathbf{x} = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0]$
 - $\mathbf{y} = [0.0, 1.0, 4.0, 9.0, 16.0, 25.0]$ (quadratic)
 - `bandwidth` = 0.5
- Action:
 - Fit LOESS model.
 - Predict at $\mathbf{x} = 2.5$.
- Expected:
 - Prediction approximates local quadratic behavior.
 - Acceptable absolute error compared to $y = x^2$, tolerance < 0.5 .

1.7.3 Test 3: Behavior at Boundaries

- Input:
 - Same as Test 2.
- Action:
 - Predict at $\mathbf{x} = 0.0$ and $\mathbf{x} = 5.0$.
- Expected:
 - No crash or invalid memory access.
 - Predictions within reasonable extrapolation range (tolerance < 1.0).

1.7.4 Test 4: Sorting Stability

- Input:
 - $x = [5.0, 1.0, 4.0, 2.0, 3.0]$ (unsorted)
 - $y = [10.0, 2.0, 8.0, 4.0, 6.0]$
 - `bandwidth = 1.0`
- Action:
 - Fit LOESS model.
 - Predict at $x = 1.0$ to 5.0 .
- Expected:
 - Sorting is correctly handled internally.
 - Predictions match original linear relationship.
 - Maximum absolute prediction error $< 10^{-12}$.

1.7.5 Test 5: Consistency Under Different Bandwidths

- Input:
 - Simple linear data as in Test 1.
- Action:
 - Fit LOESS models with bandwidths 0.4, 0.6, 0.8, 1.0.
 - Predict at $x = 3.0$.
- Expected:
 - Predicted values are consistent across bandwidths (variance $< 10^{-10}$).

2 F42 Core Sorting Module Pseudocode

2.1 Module Overview

The sorting module provides type-specific, in-place sorting routines for 1D arrays of integers, real numbers, and character sequences. It uses Fortran **interface** blocks to overload a common sort function name across types. All working memory is preallocated and passed explicitly.

2.2 Type Definitions

- No new types are defined.
- Only standard Fortran arrays are used.

2.3 Public Interface

- `interface sort_array`
 - module procedure `sort_integer`, `sort_real`, `sort_character`

Each sort routine works in-place and expects preallocated working memory.

2.4 Function Signatures

- subroutine `sort_integer(array, work)`
 - integer, dimension(n), intent(inout) :: array
 - integer, dimension(n), intent(inout) :: work (temporary work array)
- subroutine `sort_real(array, work)`
 - real, dimension(n), intent(inout) :: array
 - integer, dimension(n), intent(inout) :: work
- subroutine `sort_character(array, work)`
 - character(len=*), dimension(n), intent(inout) :: array
 - integer, dimension(n), intent(inout) :: work

2.5 Algorithm: General Structure (All Types)

1. Initialize permutation array:
$$\text{work}(i) = i \quad \text{for } i = 1, \dots, n$$
2. Sort the permutation array `work` based on the corresponding values in `array`.
 - Use insertion sort (small n) or quicksort (large n).
3. Rearrange `array` according to the sorted permutation:
 - Use a cycle leader algorithm to avoid extra memory allocation.

2.6 Details for Each Type

- **Integer:** Compare `array(i)` and `array(j)` numerically.
- **Real:** Compare `array(i)` and `array(j)` numerically.
- **Character:** Compare `array(i)` and `array(j)` lexicographically.

2.7 Memory Requirements

- For sorting an array of size n :
 - One integer work array of size n is needed.

2.8 Algorithm: Non-Recursive Quicksort for Permutation Array

To comply with F42 and WebAssembly restrictions, Quicksort is implemented without recursion, using an explicit manual stack structure for subarray boundaries.

2.8.1 Procedure: `quicksort_nonrecursive`

- subroutine `quicksort_nonrecursive(array, perm, n, stack_left, stack_right)`
 - type depends on context :: array (integer, real, or character array)
 - integer, dimension(n), intent(inout) :: perm (permutation array)
 - integer, intent(in) :: n (size of array)
 - integer, dimension(:), intent(inout) :: stack_left, stack_right (manual stack arrays)

2.8.2 Working Memory Requirements

- `stack_left` and `stack_right`: integer arrays of length at least $\log_2(n) + 10$ (safe overestimate).

2.8.3 Steps

1. Initialize:

- Set `stack_ptr = 1`.
- Push initial full range: `stack_left(1) = 1, stack_right(1) = n`.

2. Main loop:

- While `stack_ptr > 0`:
 - (a) Pop:
 - Set `left = stack_left(stack_ptr), right = stack_right(stack_ptr)`.
 - Decrement `stack_ptr`.
 - (b) If `left ≥ right`, continue to next loop iteration.
 - (c) Select pivot:
 - Set `pivot_index = (left + right) / 2`.
 - Set `pivot_value = array(perm(pivot_index))`.
 - (d) Partition:
 - Initialize `i = left, j = right`.
 - While `i ≤ j`:
 - * Increment `i` while `array(perm(i)) < pivot_value`.
 - * Decrement `j` while `array(perm(j)) > pivot_value`.
 - * If `i ≤ j`:
 - Swap `perm(i)` and `perm(j)`.
 - Increment `i`, decrement `j`.
 - (e) Push new ranges onto the stack:
 - If `left < j`:
 - * Increment `stack_ptr`.
 - * Set `stack_left(stack_ptr) = left, stack_right(stack_ptr) = j`.
 - If `i < right`:
 - * Increment `stack_ptr`.
 - * Set `stack_left(stack_ptr) = i, stack_right(stack_ptr) = right`.

2.8.4 Notes

- No recursion: stack is manually managed.
- Memory predictable: all working memory passed as arguments.
- Stateless: no hidden or module-wide variables.
- Safe for WebAssembly compilation and fully F42 compliant.

3 Unit Tests for F42 Core Sorting Module

The following unit tests verify correctness, robustness, and F42 compliance of the non-recursive sorting routines for integer, real, and character arrays. Each test assumes that all required working memory is preallocated before calling sort procedures.

3.1 Test 1: Sort Ascending Integer Array

- Input:
 - `array = [5, 2, 9, 1, 6]`
 - `work = preallocated integer array of size 5`
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array = [1, 2, 5, 6, 9]`

3.2 Test 2: Sort Descending Real Array

- Input:
 - `array = [3.5, 2.2, 8.8, 1.1, 7.7]`
 - `work = preallocated integer array of size 5`
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array = [1.1, 2.2, 3.5, 7.7, 8.8]`

3.3 Test 3: Sort Random Character Array

- Input:
 - `array = ["dog", "apple", "zebra", "cat", "bird"]`
 - `work = preallocated integer array of size 5`
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array = ["apple", "bird", "cat", "dog", "zebra"]`

3.4 Test 4: Stability on Already Sorted Arrays

- Input:
 - `array = [1, 2, 3, 4, 5]`
 - `work = preallocated integer array of size 5`
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array` remains `[1, 2, 3, 4, 5]`

3.5 Test 5: Behavior on Empty Array

- Input:
 - `array = []`
 - `work` = preallocated integer array of size 0
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - No error or crash; operation completes trivially.

3.6 Test 6: Correctness of Sorting After Random Shuffles

- Input:
 - Generate random permutation of integers $[1,2,3,\dots,1000]$.
 - `array` = shuffled integers
 - `work` = preallocated integer array of size 1000
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array` = $[1, 2, 3, \dots, 1000]$